

The URL matcher supports a popular syntax for pattern matching called RegEx (or Regular Expressions) that can be used to describe all kinds of patterns and text.

In fact even the simple example (“test/”) we used is a RegEx pattern that can match a whole range of URLs! First, notice that it begins with the caret symbol “^” and that is noware to be seen in our URL. In RegEx this symbol stands for the starting of the text to be matched. There is also a symbol for the end of the text, and that is represented by the dollar symbol “\$”. Note that we have not used that symbol in our pattern.

So the pattern “test/” means: Match any URL path that starts with ‘test/’. So it need only start with ‘test/’, you can enter anything else after that and it will still match that pattern and display that same message! For instance the URL ‘<http://localhost:8000/test/abc123>’ will match this patten and return the same message. Go ahead, try it.

Note that these patterns only match the URL path after the host name. So in the URL ‘<http://localhost:8000/some/path/>’ the beginning ‘<http://localhost:8000/>’ bit is never part of the pattern. So in the future we will leave out this bit of the URL for brevity since it is inconsequential to URL matching.

Now if you add a ‘\$’ at the end of this patterns and make it “test/\$” it will stop matching anything other than just ‘<http://localhost:8000/test/>’. This is because the pattern now means: Match text that contains ‘test/’ between its starting and ending. This of course is only true for that one URL.

On the other hand if you remove even the “^” in the beginning, it will match any URL that contains ‘test/’ anywhere in it. So it will match all of the following:

- [abctest/](#)
- [123test/](#)
- [admin/test/](#)
- [admin/edit/test/](#)

The last two of the above cases should be especially worrying since they normally would lead to your admin panel. This matching happens in order, so if a URL matches multiple patterns, the first view function is the one that will be called, and its response retured to the user.

RegEx supports matching some very complex patterns, but we wont need a lot of that here. What we will need is the ability to match year / month / day numbers, category / tag / article / gallery / image names etc. All of this is easily possible with RegEx.

If you want to match a multiple characters, you can specify them as a